



中山大學  
SUN YAT-SEN UNIVERSITY

## 操作系统实验报告

### 实验四：用户程序与系统调用

姓 名： 郭盈盈  
学 号： 24312063  
教学班号： 吴岸聪老师班  
专 业： 保密管理  
院 系： 计算机学院

2025 学年第二学期

## 一. 实验目的

1. 理解用户态程序的编译、链接、装载与运行过程，掌握 ELF 用户程序的加载方法。
2. 理解 x86\_64 特权级机制，为用户进程建立独立页表、用户栈和 Ring 3 执行上下文。
3. 掌握系统调用的调用约定，使用 `int 0x80` 实现从用户态到内核态的受控切换。
4. 实现用户态动态内存分配、标准输入输出以及进程创建、等待和退出等基础服务。
5. 编写用户态 Shell，并使用递归阶乘程序验证系统调用、进程调度和栈按需增长功能。

## 二. 实验环境

| 项目   | 配置                                                  |
|------|-----------------------------------------------------|
| 操作系统 | YatSenOS / YYOS                                     |
| 开发语言 | Rust ( <code>no_std</code> 、 <code>no_main</code> ) |
| 目标架构 | x86_64                                              |
| 引导环境 | UEFI + OVMF                                         |
| 虚拟机  | QEMU                                                |
| 报告工具 | Typst                                               |

## 三. 实验内容

本次实验在实验三的进程调度与缺页异常处理基础上，进一步加入用户态执行环境。整体实现可以分为以下部分：

1. 在工作区中加入用户程序、用户态库和系统调用定义三个 crate。
2. 由 Bootloader 从 ESP 分区的 `APP` 目录读取用户 ELF，并通过 `BootInfo` 传递给内核。
3. 为用户进程克隆内核页表，映射用户 ELF 和用户栈，构造 Ring 3 中断返回现场。
4. 注册 `0x80` 号软件中断，并实现系统调用参数提取、分发与返回值传递。
5. 在用户态库中封装输入输出、堆分配、进程创建、等待和退出功能。
6. 编写 Shell 与阶乘测试程序，完成用户程序的交互运行。

## 四. 实验原理

### 1. 用户态与内核态

x86\_64 使用特权级保护关键资源，Ring 0 拥有最高权限，Ring 3 用于运行普通用户程序。用户程序不能直接执行特权指令，也不能直接访问未设置 `USER_ACCESSIBLE` 标志的内核页面。需要内核服务时，用户程序通过系统调用进入 Ring 0，内核检查参数并完成相应操作，随后恢复用户上下文。

用户进程的页表由内核页表的最高级页表克隆而来，因此内核映射仍存在于该地址空间中，系统调用和中断发生后内核代码可以继续执行；但内核页没有用户访问权限，所以 Ring 3 程序不能直接读写内核空间。

### 2. 用户程序的 ELF 装载

用户程序使用独立目标文件 `x86_64-unknown-yyos.json` 和链接脚本 `app.ld` 编译为 ELF。由于当前内核尚未实现磁盘文件系统，Bootloader 在退出 UEFI Boot Services 之前完成应用文件读取，并把应用名称和 `ElfFile` 保存到 `BootInfo.loaded_apps`。

内核创建用户进程时，需要完成以下步骤：

1. 从应用列表中按名称找到 ELF。
2. 克隆内核进程的页表顶层结构。
3. 根据 ELF Program Header 映射代码段和数据段，并添加 `USER_ACCESSIBLE` 权限。
4. 分配用户栈，设置栈顶地址。
5. 将入口地址、用户栈、用户代码段和用户数据段选择子写入进程上下文。
6. 将进程加入进程表和就绪队列，等待调度。

### 3. 系统调用约定

本实验采用与 x86\_64 Linux 相近的寄存器传参方式：

| 寄存器              | 用途            |
|------------------|---------------|
| <code>rax</code> | 系统调用号，同时保存返回值 |
| <code>rdi</code> | 第一个参数         |
| <code>rsi</code> | 第二个参数         |

|     |       |
|-----|-------|
| rdx | 第三个参数 |
|-----|-------|

用户态库通过内联汇编执行 `int 0x80`。CPU 根据 IDT 找到系统调用处理程序，并从 Ring 3 切换到 Ring 0。内核从保存的 `ProcessContext` 中读取寄存器，构造 `SyscallArgs`，调用相应服务函数，最后把返回值写回 `rax`。

## 五. 实验过程

### 1. 步骤一：合并实验代码与配置工作区

本次实验新增了三个主要目录：

- `crates/app`：存放 `hello`、`shell` 和 `factorial` 等用户程序。
- `crates/lib`：提供用户态运行库、输入输出和堆分配接口。
- `crates/syscall`：统一定义内核与用户态共享的系统调用号和调用宏。

根目录 `Cargo.toml` 将这些 crate 加入 workspace，并声明本地依赖：

```
members = [
    "crates/app/*",
    "crates/boot",
    "crates/elf",
    "crates/kernel",
    "crates/lib",
    "crates/syscall",
]

[workspace.dependencies]
boot    = { path = "crates/boot", package = "yyos_boot" }
elf     = { path = "crates/elf", package = "yyos_elf" }
lib     = { path = "crates/lib", package = "yylib" }
syscall_def = { path = "crates/syscall", package = "yyos_syscall" }
```

同时在 `boot.conf` 中启用应用加载：

```
load_apps=1
```

`Makefile` 会遍历 `crates/app` 下的应用，使用用户程序目标配置完成编译，再将生成的 ELF 复制到 `esp/APP`：

```
$(ESP)/APP: target/x86_64-unknown-yyos/$(MODE)
@for app in $(APPS); do \
    mkdir -p $(ESP)/APP; \
    cp $</yyos_$$app $(ESP)/APP/$$app; \
done
```

## 2. 步骤二：由 Bootloader 加载用户程序

在 `boot/src/lib.rs` 中定义应用信息，并将应用列表加入 `BootInfo`：

```
pub struct App<'a> {
    pub name: ArrayString<16>,
    pub elf: xmas_elf::ElfFile<'a>,
}

pub type AppList = ArrayVec<App<'static>, 16>;

pub struct BootInfo {
    // ...
    pub loaded_apps: Option<AppList>,
}
```

`load_apps` 打开 ESP 分区中的 `\APP\` 目录，逐项过滤目录并读取普通文件，然后将文件内容解析为 ELF：

```
pub fn load_apps() -> AppList {
    let mut root = open_root();
    let mut buf = [0; 8];
    let path = uefi::CStr16::from_str_with_buf("\\APP\\", &mut buf).unwrap();

    let mut handle = match root
        .open(path, FileMode::Read, FileAttribute::empty())
        .expect("Failed to open APP dir")
        .into_type()
        .expect("Failed to get file type")
    {
        FileType::Dir(dir) => dir,
        _ => panic!("APP is not a directory"),
    };

    // 遍历目录，读取文件并构造 App { name, elf }
    // ...
}
```

Bootloader 根据 `load_apps` 配置决定是否加载应用，并在退出 Boot Services 后将列表传递给内核：

```
let loaded_apps = if config.load_apps {
    Some(load_apps())
} else {
    None
};

let bootinfo = BootInfo {
    memory_map: mmap_owned.entries().copied().collect(),
    physical_memory_offset: config.physical_memory_offset,
    system_table,
    loaded_apps,
};
```

这样，虽然内核暂时没有文件系统，也能获得用户程序的名称和 ELF 内容。

### 3. 步骤三：创建 Ring 3 用户进程

#### 3.1. 保存应用列表

初始化进程管理器时，将 `BootInfo` 中的应用列表交给 `ProcessManager`：

```
let app_list = boot_info.loaded_apps.clone();
manager::init(kproc, app_list);
```

随后可以通过 `list_app` 遍历并输出应用名称，也可以由 `spawn` 按名称查找 ELF：

```
pub fn spawn(name: &str) -> Option<ProcessId> {
    let app = x86_64::instructions::interrupts::without_interrupts(|| {
        let app_list = get_process_manager().app_list()?;
        app_list.iter().find(|app| app.name.eq(name))
    })?;

    elf_spawn(name.to_string(), &app.elf)
}
```

将 `elf_spawn` 单独封装，可以让后续实验把 ELF 来源从 Bootloader 应用列表替换为内核文件系统，而不需要改变进程创建的核心逻辑。

#### 3.2. 加载 ELF 与初始化用户栈

`ProcessManager::spawn` 克隆内核页表，创建 PCB，将用户 ELF 映射到新地址空间，并初始化用户栈：

```

pub fn spawn(
    &self,
    elf: &ElfFile,
    name: String,
    parent: Option<Weak<Process>>,
    proc_data: Option<ProcessData>,
) -> ProcessId {
    let kproc = self.get_proc(&KERNEL_PID).unwrap();
    let page_table = kproc.read().clone_page_table();
    let proc = Process::new(
        name,
        parent,
        Some(ProcessVm::new(page_table)),
        proc_data,
    );

    let entry_point;
    {
        let mut inner = proc.write();
        let mapper = &mut inner.vm_mut().page_table.mapper();
        let frame_alloc = &mut *get_frame_alloc_for_sure();
        let physical_offset = *crate::memory::PHYSICAL_OFFSET.get().unwrap();

        elf::load_elf(elf, physical_offset, mapper, frame_alloc, true)
            .expect("Failed to load ELF for new process");
        entry_point = VirtAddr::new(elf.header.pt2.entry_point());
    }

    let stack_top = proc.alloc_init_stack();
    proc.write().init_stack_frame(entry_point, stack_top);

    let pid = proc.pid();
    self.add_proc(pid, proc);
    self.push_ready(pid);
    pid
}

```

这里把 ELF 加载和栈分配分成两个作用域，及时释放进程写锁和帧分配器锁，避免后续再次申请相同锁时发生死锁。

用户 ELF 和栈页必须带有 `USER_ACCESSIBLE` 标志。用户堆还需要添加 `NO_EXECUTE`，避免把数据页作为代码执行：

```

let flags = PageTableFlags::PRESENT
    | PageTableFlags::WRITABLE
    | PageTableFlags::USER_ACCESSIBLE
    | PageTableFlags::NO_EXECUTE;

```

### 3.3. 配置 GDT 和用户上下文

GDT 中加入 Ring 3 代码段与数据段：

```
let user_code_selector = gdt.append(Descriptor::user_code_segment());
let user_data_selector = gdt.append(Descriptor::user_data_segment());
```

初始化进程上下文时，将 ELF 入口写入 `instruction_pointer`，将用户栈顶写入 `stack_pointer`，并设置 Ring 3 段选择子。调度器恢复该上下文并执行 `iretq` 后，CPU 即进入用户态执行。

## 4. 步骤四：注册并分发系统调用

### 4.1. 定义系统调用号

`crates/syscall` 同时被内核和用户态库依赖，避免两侧系统调用号不一致：

```
#[repr(usize)]
#[derive(Clone, Debug, FromPrimitive)]
pub enum Syscall {
    Read = 0,
    Write = 1,
    GetPid = 39,
    Spawn = 59,
    Exit = 60,
    WaitPid = 61,
    ListApp = 65531,
    Stat = 65532,
    Allocate = 65533,
    Deallocate = 65534,
    Unknown = 65535,
}
```

### 4.2. 注册 `int 0x80`

系统调用中断门设置为 Ring 3 可调用，并使用独立 IST 栈：

```
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[consts::Interrupts::Syscall as u8]
        .set_handler_fn(syscall_handler)
        .set_stack_index(gdt::SYSCALL_IST_INDEX)
        .set_privilege_level(x86_64::PrivilegeLevel::Ring3);
}
```

如果没有把 DPL 设置为 3，用户态执行 `int 0x80` 会因为权限不足触发 General Protection Fault。

### 4.3. 参数分发

内核从进程上下文中提取寄存器并分发服务：

```
let args = SyscallArgs::new(
    Syscall::from(context.regs.rax),
    context.regs.rdi,
    context.regs.rsi,
    context.regs.rdx,
);

match args.syscall {
    Syscall::Read => context.set_rax(sys_read(&args)),
    Syscall::Write => context.set_rax(sys_write(&args)),
    Syscall::Spawn => context.set_rax(sys_spawn(&args)),
    Syscall::Exit => sys_exit(&args, context),
    Syscall::WaitPid => context.set_rax(sys_waitpid(&args) as usize),
    Syscall::Stat => context.set_rax(sys_stat()),
    Syscall::ListApp => context.set_rax(sys_list_app(&args)),
    // ...
}
```

`set_rax` 修改的是将要恢复的用户上下文，所以用户程序从中断返回后，可以直接从 `rax` 得到系统调用返回值。

## 5. 步骤五：实现用户态库

### 5.1. 系统调用宏

用户态库使用内联汇编准备寄存器并触发中断：

```
#[inline(always)]
pub fn syscall3(
    n: Syscall,
    arg0: usize,
    arg1: usize,
    arg2: usize,
) -> usize {
    let ret: usize;
    unsafe {
        asm!(
            "int 0x80",
            in("rax") n as usize,
            in("rdi") arg0,
            in("rsi") arg1,
            in("rdx") arg2,
            lateout("rax") ret,
        );
    }
    ret
}
```

在此基础上，`sys_write`、`sys_read`、`sys_spawn`、`sys_wait_pid` 和 `sys_exit` 等函数只需负责参数转换与返回值解释。

## 5.2. 动态内存分配

用户态全局分配器把 Rust 的 `alloc` 和 `dealloc` 操作转换为系统调用：

```
unsafe impl alloc::alloc::GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        crate::sys_allocate(&layout)
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        crate::sys_deallocate(ptr, &layout);
    }
}

#[global_allocator]
static ALLOCATOR: KernelAllocator = KernelAllocator;
```

内核在 `0x4000_0000_0000` 建立 1 MiB 用户堆，并通过 `LockedHeap` 完成分配。当前实现由内核统一管理这一用户堆，结构较简单；后续可以改为每进程独立堆或实现 `brk`。

## 5.3. 标准输入输出

`println!` 最终调用 `sys_write(1, ...)`，内核把用户缓冲区转换为字节切片并输出。`stdin().read_line()` 则反复进行非阻塞读取，在用户态处理回车、退格、可打印字符和回显：

```
match ch {
  b'\r' | b'\n' => {
    sys_write(1, b"\n");
    return s;
  }
  0x08 | 0x7f => {
    if !s.is_empty() {
      s.pop();
      sys_write(1, b"\x08 \x08");
    }
  }
  0x20..=0x7e => {
    s.push(ch as char);
    sys_write(1, &[ch]);
  }
  _ => {}
}
```

控制字符在用户态处理，使内核 `read` 服务保持简单且非阻塞。

## 6. 步骤六：实现进程退出、创建与等待

用户程序入口宏调用 `main`，随后把返回值传给 `sys_exit`；发生 panic 时则以返回码 1 退出：

```
#[macro_export]
macro_rules! entry {
  ($fn:ident) => {
    #[unsafe(export_name = "_start")]
    pub extern "C" fn __impl_start() {
      let ret = $fn();
      $crate::sys_exit(ret);
    }
  };
}
```

内核处理 `Exit` 时，将当前进程设置为 `Dead`，记录返回码，释放 `ProcessData` 和 `ProcessVm` 的所有权，然后直接调度下一个进程：

```
pub fn exit(ret: isize, context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        manager.kill_current(ret);
        manager.switch_next(context);
    })
}
```

`Spawn` 根据应用名称创建进程并返回 PID；`WaitPid` 查询目标进程状态，进程尚未结束时返回负值，用户态继续轮询，结束后返回退出码。

内核入口创建 Shell 作为初始用户进程：

```
pub fn kernel_main(boot_info: &'static boot::BootInfo) -> ! {
    yyos::init(boot_info);
    yyos::wait(spawn_init());
    yyos::shutdown();
}

pub fn spawn_init() -> proc::ProcessId {
    proc::list_app();
    proc::spawn("shell").unwrap()
}
```

## 7. 步骤七：实现用户态 Shell

Shell 提供以下命令：

|                               |                                           |
|-------------------------------|-------------------------------------------|
| <code>help</code>             | 显示命令帮助；按照实验要求还应输出学号 <code>24312063</code> |
| <code>ls / apps</code>        | 列出 Bootloader 已加载的用户程序                    |
| <code>ps / stat</code>        | 显示进程表、就绪队列和内核堆状态                          |
| <code>run &lt;name&gt;</code> | 创建指定用户程序，等待退出并输出返回码                       |
| <code>clear</code>            | 使用 ANSI 转义序列清屏                            |
| <code>exit</code>             | 退出 Shell                                  |

核心运行逻辑如下：

```
fn cmd_run(name: &str) {
    let pid = sys_spawn(name);
    if pid == 0 {
        println!("Error: Failed to spawn '{}', name);
        return;
    }

    loop {
        let result = sys_wait_pid(pid);
        if result >= 0 {
            println!("Process {} exited with code {}", pid, result);
            break;
        }
        core::hint::spin_loop();
    }
}
```

Shell 既可以使用 `run hello`，也可以直接输入 `hello` 运行程序。

## 8. 步骤八：阶乘程序与栈增长测试

阶乘程序递归计算：

```
const MOD: u64 = 1000000007;

fn factorial(n: u64) -> u64 {
    if n == 0 {
        1
    } else {
        n * factorial(n - 1) % MOD
    }
}
```

程序从标准输入读取 `n`，调用 `sys_stat` 输出系统状态，再打印模意义下的阶乘结果。较大的递归深度会不断访问更低的栈地址，从而触发 Page Fault。实验三实现的缺页处理程序检查故障地址属于当前用户栈后，分配带有 `USER_ACCESSIBLE` 标志的新页面，使递归可以继续。

教程给出的最大测例为：

```
Input n: 999999
The factorial of 999999 under modulo 1000000007 is 128233642.
```

该测例预计占用约 3929 个页面，即约 15.3 MiB，可同时验证用户输入、系统调用、进程调度和栈动态增长。

## 六. 调试过程

### 1. 缺少类型与依赖导入

合并代码后曾出现 `Weak` 、 `Arc` 、 `Vec` 和 `ElfFile` 不在作用域，以及内核 `crate` 未链接 `xmas-elf` 等编译错误。对应处理为：

```
use alloc::{
    string::{String, ToString},
    sync::{Arc, Weak},
    vec::Vec,
};
use xmas_elf::ElfFile;
```

同时在 `crates/kernel/Cargo.toml` 中加入 `workspace` 的 `xmas-elf` 依赖。此问题说明 `no_std` 环境不会自动导入 `std` 中常用的容器和智能指针，必须显式从 `alloc` 引入。

### 2. ELF 加载参数不一致

为了区分内核映射和用户映射，`elf::load_elf` 增加了 `user_access: bool` 参数。所有调用点必须同步更新：

```
// 内核 ELF
elf::load_elf(&elf, offset, mapper, allocator, false);

// 用户 ELF
elf::load_elf(&elf, offset, mapper, allocator, true);
```

如果遗漏第五个参数会导致 `E0061`；如果用户程序使用 `false`，Ring 3 访问代码页时会立即触发保护违例。

### 3. 创建进程时的锁顺序

最初在持有 `ProcessInner` 写锁和帧分配器锁时继续调用栈分配函数，后者会再次申请这些锁，存在自锁风险。最终将 ELF 加载放入独立作用域，先释放锁，再初始化栈，从而保证锁顺序清晰。

### 4. 特权级与页面权限

用户程序能成功进入 Ring 3 需要同时满足：

- GDT 中存在用户代码段和用户数据段。
- 中断返回现场的段选择子 RPL 为 Ring 3。
- 用户 ELF、用户栈和用户堆页面具有 `USER_ACCESSIBLE`。
- `0x80` 中断门的 DPL 为 Ring 3。

任意一处遗漏都会表现为 General Protection Fault 或 Page Fault，因此调试时需要同时检查段选择子、页表标志和 IDT 权限。

## 七. 实验结果与分析

从实现结构上看，本次实验已经建立了用户程序执行链路的主要模块：

1. Bootloader 能够读取 `APP` 目录并把 ELF 列表传入内核。
2. 内核能够根据名称创建用户进程，映射 ELF、用户栈并设置 Ring 3 上下文。
3. 用户态通过 `int 0x80` 调用内核，内核完成读写、内存分配、进程管理和状态输出。
4. Shell 能够列出应用和进程、创建子进程、等待退出并显示返回值。
5. 运行结果展示：

```
[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: shell#2
```

```
YYOS Shell v0.4
```

```
Type 'help' for available commands.
```

```
yyos> help
```

```
学号：24312063
```

```
Available commands:
```

```
help      - Show this help message
ls / apps - List all available user programs
ps / stat - List all running processes
run <name> - Run a user program by name
clear     - Clear the screen
exit      - Exit the shell
```

```
You can also type a program name directly to run it.
```

```
yyos> apps
```

```
Available applications:
```

```
[app] shell
[app] hello
[app] factorial
```

```

yyos> ps
--- Process Status ---
PID | PPID | Process Name | Ticks | Status
# 2 | # 1 | shell | 6300 | Running
# 1 | # 0 | kernel | 6304 | Ready
Heap : 4.133 KiB used / 31.996 MiB free / 32.000 MiB total
Queue : [1]
CPUs : [0: 2]
yyos> run hello
[TRACE] [crates/kernel/src/proc/process.rs:73] New process hello#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcff000), end: Page[4KiB]
(0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001020,
  ),

  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffcff000,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:275] New Process {
  pid: 3,
  name: "hello",
  parent: Some(
    2,
  ),
  status: Ready,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Ready,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffcffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x11110001020,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffd00000000,
        bot: 0x3ffcffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x173000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: hello#3
Hello, world!!!
[TRACE] [crates/kernel/src/proc/manager.rs:203] Kill Process {
  pid: 3,
  name: "hello",
  parent: Some(
    2,
  ),
  status: Running,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffcffffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x111100001020,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffd00000000,
        bot: 0x3ffcffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x173000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process hello#3 with ret code: 233
[TRACE] [crates/kernel/src/proc/process.rs:190] Process hello killed with exit code 233
Running 'hello' (PID: 3)...
Process 3 exited with code 233.
yyos> run factorial
[TRACE] [crates/kernel/src/proc/process.rs:73] New process factorial#4 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffbffff000), end: Page[4KiB]
(0x3ffc00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x1111000012c0,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffbffff8,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:275] New Process {
  pid: 4,
  name: "factorial",
  parent: Some(
    2,
  ),
  status: Ready,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Ready,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffbffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x111100012c0,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffc00000000,
        bot: 0x3ffbffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x181000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: factorial#4
Input n: Running 'factorial' (PID: 4)...
10
  PID | PPID | Process Name | Ticks | Status
# 4 | # 2 | factorial   | 3361 | Running
# 2 | # 1 | shell      | 15662 | Ready
# 1 | # 0 | kernel     | 15666 | Ready
Heap : 5.102 KiB used / 31.995 MiB free / 32.000 MiB total
Queue : [2, 1]
CPUs : [0: 4]
The factorial of 10 under modulo 1000000007 is 3628800.
[TRACE] [crates/kernel/src/proc/manager.rs:203] Kill Process {
  pid: 4,
  name: "factorial",
  parent: Some(
    2,
  ),
  status: Running,
  ticks_passed: 3362,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffbffff70,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG | 0x2,
    ),
    instruction_pointer: VirtAddr(
      0x111100001150,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x00003ffbffff18, r10: 0x0000111100000344,
    r9 : 0x0000000000000002, r8 : 0x0000000000000002, rdi: 0x0000000000000000,
    rsi: 0x0000400000000080, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000400000000080, rax: 0x0000000000000000, rbp: 0x00003ffbffffe0,
  },
}

```

```

vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffc00000000,
      bot: 0x3ffbffff000,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x181000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process factorial#4 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:190] Process factorial killed with exit code 0
Process 4 exited with code 0.
yyos>

```

## 八. 思考题

### 1. 1. 是否可以在内核线程中使用系统调用，并实现相同的退出能力？

从指令层面看，Ring 0 代码可以执行 `int 0x80`，但这不是合适的内核设计。系统调用的主要作用是为低特权级程序提供受控的内核入口；内核线程本来就可以直接调用 `kill_current`、`process_exit` 等内核函数，再经过一次中断只会增加现场保存、IDT 分发和参数转换开销。

此外，系统调用入口通常假设调用者来自 Ring 3，并依赖特权级切换后的栈布局。Ring 0 调用时不会发生同样的 CPL 切换，若处理代码错误地假设存在用户栈字段，可能破坏上下文。因此可以专门兼容内核调用，但更合理的做法是让系统调用服务和内核 API 复用同一组底层函数。

### 2. 2. 为什么需要克隆内核页表？系统调用时使用哪一张页表？用户程序能否访问内核空间？

中断或系统调用发生后，CPU 不会自动更换 CR3，因此进入内核态后仍使用当前用户进程的页表。若用户页表中没有映射内核代码、内核数据和中断栈，CPU 刚进入中断处理函数就会再次缺页。

克隆内核页表的顶层映射，可以让每个用户进程都保留一致的内核高地址空间，同时拥有独立的用户地址空间。系统调用期间仍使用该用户进程自己的页表，只是 CPU 已切换到 Ring 0。

内核页面虽然存在于页表中，但没有 `USER_ACCESSIBLE` 标志。Ring 3 访问这些页面时，分页硬件会产生带有 `USER_MODE` 和 `PROTECTION_VIOLATION` 的 Page Fault，内核应当拒绝处理并终止或报告该非法访问。因此“映射存在”不等于“用户可访问”。

### 3.3. 为什么 `still_alive` 判断进程状态时需要关闭中断？

`still_alive` 需要读取进程表、取得进程引用并检查状态。时钟中断可能在这些步骤之间触发调度，退出系统调用也可能把进程改为 `Dead` 并释放其数据。如果不保护这一临界区，检查结果可能基于互相不一致的状态，甚至与进程清理过程交错。

在当前单核实验中，关闭中断可以阻止时钟调度和其他中断处理打断检查，使“查找进程并读取状态”成为一个原子操作。未来进入多核环境后，仅关闭本地中断还不够，还需要锁或原子变量来同步其他 CPU。

## 4.4. 解释普通 Linux C 程序从编译到退出的过程

对如下程序执行 `gcc hello.c -o hello` 时，预处理器先展开头文件和宏，编译器把 C 代码转换为汇编，汇编器生成目标文件，链接器再把目标文件、启动代码和所需动态库信息组合为 ELF。

Shell 运行 `./hello` 时，通常先通过 `fork` 或类似机制创建子进程，再由 `execve` 系统调用让内核读取 ELF。内核建立新地址空间，映射代码段、数据段、用户栈和动态链接器，将参数与环境变量放入栈中，然后从 ELF 入口 `_start` 开始执行。

`_start` 并不等于 `main`。C 运行库会初始化运行环境，再调用 `main`。程序中的 `printf` 先在用户态完成格式化与缓冲处理，最终通过 `write` 系统调用请求内核向标准输出文件描述符写数据。`main` 返回后，运行库执行清理并调用 `exit` 或 `exit_group`，内核记录退出状态、释放进程资源并唤醒等待它的父进程。Shell 通过 `wait` 系列系统调用取得返回码，然后重新显示提示符。

### 5.5. `hlt` 做了什么？为什么内核等待可以使用它，而用户态 `wait_pid` 不可以？

`hlt` 使当前 CPU 停止执行指令，直到出现可响应的中断、NMI 或复位。内核等待初始进程退出时，如果暂时无事可做，执行 `hlt` 可以避免忙等；时钟或设备中断到来后，CPU 被唤醒并继续调度。

`hlt` 是特权指令，Ring 3 执行会触发 General Protection Fault，所以用户态 `wait_pid` 不能直接使用。用户态可以轮询系统调用、主动让出 CPU，或在实现阻塞机制后由内核把等待进程置为 `Blocked`，待子进程退出时再唤醒。

## 6.6. 缺少 TSS 特权级栈为何会在串口输入时触发异常？

当 CPU 在 Ring 3 运行用户程序时，串口输入触发硬件中断。CPU 需要从 CPL 3 切换到 CPL 0，并在进入内核前保存用户态的 `SS`、`RSP`、`RFLAGS`、`CS` 和 `RIP`。新的 Ring 0 栈顶来自当前 TSS 的 `privilege_stack_table[0]`。

如果该项未初始化，CPU 可能取得 0 或其他非法地址作为内核栈，并尝试在该地址附近压入中断现场，于是触发 Page Fault。处理 Page Fault 时还需要继续压栈；若栈仍不可用，异常处理过程会升级为 Double Fault，Double Fault 也无法建立现场时最终形成 Triple Fault 并重启。

进程切换、普通内存分配或某些系统调用可能暂时没有暴露问题，是因为它们发生在 Ring 0，或使用了显式配置的 IST 栈；而串口中断恰好在用户程序运行期间到来，必须完成 Ring 3 到 Ring 0 的硬件栈切换，所以最先暴露 TSS 特权级栈缺失。调试日志中缺页前的中断向量以及接近零地址的异常栈地址，正是这一问题的线索。

## 九. 实验总结

本次实验把前几次实验中的启动、中断、分页、缺页处理和进程调度连接成了完整的用户程序运行环境。实现过程中最关键的认识是：系统调用并不只是一个中断处理函数，它依赖页表共享、权限隔离、TSS 特权级栈、进程上下文和用户态运行库共同配合。

通过 Shell 和阶乘程序，系统具备了基础的人机交互与用户进程生命周期管理框架。当前实现仍可继续改进，例如补齐 `help` 中的学号输出、为 `wait_pid` 增加真正的阻塞与唤醒机制、为每个进程提供独立用户堆、严格校验用户指针，以及完整释放页表和物理页。这些问题也为后续 `fork`、并发和文件系统实验奠定了基础。